

■■ METAMARK — FULL QA / SECURITY / FUNCTIONALITY AUDIT

Verdict: ■ FUNCTIONAL PROTOTYPE — SIGNIFICANT GAPS **Overall Score:** 34 / 100

Executed Environment

- Python 3.13, Node 22, MySQL 8.0 (port 3306), Google Chrome (headless)
- Real live network to Amazon.in and a **live Google Gemini 2.5 Flash API key** (from `.env`)

Constraint disclosure: The MySQL `amazon_scraper_db` could NOT be brought online because the documented `.env` (`127.0.0.1:3307`, user `metamark`, password `metamark#2026`) does not match the machine's running MySQL (port **3306**, that user/password **denied**, and the `amazon_scraper_db` **database does not exist**). This is itself a primary finding. Because of this, login/sign-up, product-persistence, and heatmap DB-backed flows could not be exercised end-to-end against a live DB. Every other pipeline (AI, OCR, compliance, scraper) was tested by calling the modules directly with live Gemini and live Amazon.

PHASE 1 — FEATURE MATRIX (Verified)

FEATURE	DOCUMENTED	IMPLEMENTED?	ACTUALLY WORKS?	EVIDENCE
Next.js/React frontend	Yes	■ Yes	■ Builds (14 routes)	<code>npm run build</code> success
Flask backend	Yes	■ Yes	■■ Partial (DB unreachable)	import OK; health returns 200 but <code>database:"unavailable"</code>
MySQL storage	Yes	■■ Schema present	■ No (not initialized; schema out of sync)	<code>amazon_scraper_db</code> absent; <code>.env</code> port 3307 vs real 3306
Gemini integration	Yes	■ Yes	■ Genuinely calls Gemini	Live API returned real OCR/text analysis (~43s)
OCR (multimodal vision)	Yes	■ Yes	■ Works live	Extracted 9 items / 307 chars from generated label
Compliance/rules engine	Yes	■ Yes (hardcoded dicts)	■■ Buggy	False-positive country mismatch; Windows crash
RAG / vector DB	"rule retrieval"	■ Not implemented	■ No	No vectorstore/embeddings/FAISS/Chroma anywhere; rules are hardcoded dicts in prompts
Web scraper (Selenium/BS)	Yes	■ Yes	■ Works live	Fetches 2.1MB Amazon HTML, extracted real product (title/price/24 images)

FEATURE	DOCUMENTED	IMPLEMENTED?	ACTUALLY WORKS?	EVIDENCE
Product analysis pipeline	Yes	■ Yes	■■ Partial (crashes on Windows)	UnicodeEncodeError mid-pipeline
Report generation	Yes	■ Yes	■■ Partial (score+grade+fake violation)	Generated grade A report w/ wrong high-priority violation
Heatmap	Yes	■ Yes (UI/API)	■ Not working	api/global-heatmap unauthenticated; maps key is a Gemini key , not a Maps key → map won't load
Chatbot	Yes	■ Yes (LangChain SQL agent)	■■ Partial (auth key mismatch + XSS)	reads <code>userid</code> vs login writes <code>user_id</code> ; <code>dangerouslySetInnerHTML</code>
Meta-Token rewards	Yes	■■ Endpoints exist	■ Broken + exploitable	self-mint <code>/add-tokens</code> ; <code>users</code> table has no <code>mt_tokens</code> ; no gifts tables
Chrome extension	Yes	■ Yes (Amazon)	■■ Partial; Flipkart fake	prod-page detection works; backend rejects Flipkart ("Invalid Amazon URL"); demo mode not shown
Demo mode	Yes	■ Yes	■ Works (clearly tagged) — but frontend ignores tag	<code>demo_compliance_report()</code> fixed 70/B, <code>demo_mode:True</code>
Env variables	Yes	■ <code>.env</code> loaded	■■ Stale/mismatched	port 3307 vs 3306; wrong DB password
External deps	Yes	■ Installed	■	imports OK; Selenium not installed (browser fallback dead)

PHASE 2 — INSTALL & RUN (What a new dev sees)

Step	Result
Python deps	■ <code>venv</code> imports fine
Node deps	■ installed
Frontend build	■ success
Backend startup	■■ starts, but every DB call fails
MySQL	■ <code>.env</code> says port 3307; real MySQL is 3306; <code>metamark</code> user/password denied; <code>amazon_scraper_db</code> absent
DB init	■ <code>database_schema.sql</code> documented at <code>server.py:2027</code> & README doesn't exist (only <code>dump.sql</code> , <code>schema.txt</code>)
Backend <code>GET /api/health</code>	■ Returns 200 "status:ok" even when "database":"unavailable" — misleading health check
Extension setup	■ Selenium/undetected-chromedriver not installed (commented out) → browser fallback dead

PHASES 3–6 — FEATURE / E2E / TEST CASES / AI VALIDATION

PHASE 6 & PHASE 4 — Is Gemini real? YES, GENUINELY.

With the live key I drove a real image + text through `analyze_seller_upload_text()`:

- Real OCR (9 items, 307 chars), real LLM data-analysis, real scoring, ~43s per analysis.

- **NOT hardcoded** in the real path. Demo mode is separate and hardcoded (clearly tagged).

PHASE 5 — REASONABLE TEST CASES THROUGH THE LIVE AI PIPELINE

#	Case	Expected	Actual	Verdict
1	Fully compliant label (Mumbai, India)	PASS / high score	Score 81.5, grade A, <code>ready:false</code> + FALSE high "Country India ≠ Address USA"	FAIL (false positive)
2	Missing MRP	Flag missing MRP	Score 61, B, caught missing MRP ■ + bogus "USA mismatch" ■	PARTIAL
3	Blurry label	Warn	Flagged missing mfr address & consumer care	PASS (reasonable)
4	Irrelevant image / deceptive claims	Reject	Grade F, score 0 — correctly rejected	■ PASS
5	Expired (Best Before 2020)	Flag expiry	Score 66, B; generic violations only	PARTIAL (missed specific expiry)
6	Prompt-injection description	Resist	Hit Gemini 429 rate limit; no graceful retry → crash	FAIL (no injection guard / no rate handling)

CRITICAL BUG #3 (reproduced 3x): Country-of-origin cross-validation false positive

`compliance_copy.py:1094` lists USA variants as `['usa', 'united states', 'america', 'us']`. The substring check `'us' in address` matches "NutriPlus" (any word containing "us"). So a fully-compliant Indian product named/manufactured by anything with "us" in it is flagged with a **HIGH-priority "Country of Origin vs Address Consistency" violation** and blocked from upload. Verified: `'us' in 'nutriplus...' → True`. This is hardcoded logic, not an LLM error, and it **misclassifies compliant products as non-compliant**.

CRITICAL BUG #2 (reproduced 3x): Windows crash

The `log()` in `compliance_copy.py:53-56` (and `compliance.py/comply.py`) `print()`s Unicode box-drawing chars (■■) which **crash on Windows cp1252 consoles** with `UnicodeEncodeError`, aborting the scoring step mid-analysis. On this Windows target, the seller/check-compliance pipeline **crashes** unless `PYTHONIOENCODING=utf-8` is set (which no user would set). This also leaks the full traceback (see security).

CRITICAL BUG #4: `analyze_product_data` crashes on `None` values

`compliance_copy.py:782` slices `value[:60]` with no `None`-guard → `TypeError` when the LLM returns a null finding.

PHASE 8 — RAG: NOT IMPLEMENTED.

Zero vector DB / embeddings / retrievers. "Rule retrieval" is just injecting a hardcoded Python dict into the Gemini prompt. Nothing to retrieve; nothing grounded; no retrieval quality to test. The LLM, not any RAG, decides (see Phase 7).

PHASE 7 — LEGAL COMPLIANCE LOGIC

- **Implemented logic:** hardcoded rule dicts (`LEGAL_METROLOGY_RULES`, `REGULATORY_RULES` with rule

numbers) injected into the prompt + naive cross-validation + penalty scoring.

- **Verified behavior:** the LLM is fully capable of inventing/mis-applying rules; the deterministic

cross-validation **falsely flags compliant products** (BUG #3). Grade/score thresholds differ between modules (`comply.py` A+ at 85 vs siblings), so **the same product gets different grades depending on which endpoint processes it**.

- ****It can mark compliant as non-compliant (proven) and, absent a real rules DB, may mark non-compliant**

as compliant. **The "rules citations" are LLM-generated and** not verified against an actual regulation database. **There is** no ground-truth authority** in the loop.

PHASE 9 — SCRAPER: SHARED FETCHER GENUINELY WORKS (Amazon)

Live test: fetched 2.1MB Amazon HTML (no CAPTCHA), extracted real title "Leriya Fashion Co-ord Set...", price ■529, ASIN `B0F6K68VL4`, seller+store URL, **24 images**. UA rotation/retries/bot-wall detection all function. A dead ASIN correctly returned 404→None. **But Flipkart is a false promise:** `content.js` detects Flipkart, but the backend `extract_asin_from_url` only accepts `/dp//gp/product/` (Amazon) → returns "Invalid Amazon URL". Selenium fallback is dead (not installed).

PHASE 10 — CHROME EXTENSION

- Amazon product-page detection ■, overlay injection ■, real backend connection ■.
- **Flipkart detection only, cannot process ■.**
- **Demo mode NOT communicated** to the user (score 70/B shown as if real) ■.
- ID/role passed in URL query params (history/log exposure) ■■.
- High: `cookies` permission declared but unused; **no URL validation on the background API proxy**

(open proxy → SSRF-ish with user cookies); `content.js` injects API data via unsanitized `innerHTML` (XSS).

PHASE 11 — DATABASE

- Schema has only 4 tables (`users`, `products`, `images`, `selleractivity`).
- **Missing:** `users.mt_tokens`, `gifts`, `gifts_redeemed`, `products.product_json_raw` — yet multiple endpoints query/write them → runtime SQL failures even with a healthy connection.
- `dump.sql` ships unsalted SHA-256 hashes of `password123/123456` for seeded users (crackable credential leak).
- No FKs enforced; duplicate/key consistency weak.

- `/api/product/<id>` and `/api/products/validate/<id>` have **no ownership check** → IDOR (any user can read/validate any product).
- Could not confirm persistence across restart because DB is unreachable; persistence is entirely DB-bound.

PHASE 12 — SECURITY (High → Low)

Severity	Issue	Location
■ CRIT	Client self-mints tokens — logged-in customer adds unlimited MT via <code>/api/gifts/add-tokens</code>	server.py:1843
■ CRIT	Unauthenticated gift list leaks <code>gift_pin</code> to everyone; <code>/api/gifts</code> , <code>/api/gifts/list</code> , <code>/api/global-heatmap</code> , <code>/api/image/<id></code> unauthenticated	server.py
■ CRIT	Full Python traceback leaked to API client (<code>/api/seller/check-upload-text</code>)	server.py:2017
■ CRIT	Plaintext DB password + live Gemini key in <code>.env</code> (gitignored, but on disk)	<code>.env</code>
■ HIGH	Extension background API proxy — no URL validation (SSRF w/ user cookies)	ext/background.js
■ HIGH	XSS via <code>dangerouslySetInnerHTML</code> on untrusted LLM content	chatbot/page.jsx:311
■ HIGH	Prompt injection not defended; malicious description reaches LLM; no guard rails	server.py/compliance
■ HIGH	Unsalted SHA-256 passwords; no logout/rate-limit	server.py:211
■ HIGH	No rate limiting anywhere ; free-tier 20 req/day quota hit in minutes	—
■ MED	CSV/CSRF limited (CORS set to localhost:3000 ■)	server.py
■ MED	Inconsistent DB-CRED URL building (no password URL-encoding) in most modules	compliance*.py

PHASE 13 — FRONTEND QA

- **Build passes** (14 static routes).
- **Bug:** login writes `user_id/user_role`, chatbot reads `userid/userrole` → `**chatbot` never sees login state** (verified lines).
- **Heatmap won't render:** `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY` is set to a **Gemini AQ...** key, not a Google Maps `AIza...` key.
- **Demo mode never surfaced** in any page/extension → users can't distinguish fake from real.
- Landing page background video `backgroundvideo4.mp4` missing (gitignored) → silent blank.
- Heavy JS: landing **764 kB** first-load (Spline/GSAP); dashboard 229 kB. Mobile responsiveness and

empty states not verifiable without a live DB.

PHASE 14 — PERFORMANCE

- Single seller-upload analysis: ~43–47s (multiple sequential Gemini calls) — too slow for interactive demo.
- Gemini **free tier: 20 requests/day, per model** → after ~6 analyses the system is rate-limited with **no retry/backoff** (crashes on `GoogleRateLimitError`).
- Scraper ~3–16s per product. No caching. Geocoding calls external `ipapi.co` per request.

PHASE 15 — DEMO MODE

- **Implemented and clearly tagged** at backend (`demo_mode: True`, explanatory messages) — good.
- **But dangerous:** demo report returns `ready_for_upload: True, is_compliant: True`, grade B for **every** product, with **no real analysis**. And the frontend/extension **ignore the flag** — so in demo mode **every product looks compliant/ready**, which could mislead sellers/consumers. That's a real integrity risk, not just a cosmetic gap.

FINAL VERDICT

■ FUNCTIONAL PROTOTYPE — SIGNIFICANT GAPS

This is **not** "does not work" — the AI and scraper genuinely function and the frontend builds. But it is **not** a production-claimable product: it crashes on Windows, produces **provably wrong compliance verdicts**, has a broken/insecure reward economy, an unrealized RAG claim, a false Flipkart promise, and critical security holes.

Overall Score: 34 / 100

Feature Status Table (summary)

Feature	Status	Evidence
Gemini OCR/Analysis	■ Real/works	Live API output
Amazon scraper	■ Real/works	Live Amazon extraction
Frontend build	■ Works	npm build
Compliance verdicts	■ WRONG	false "USA" mismatch
Seller-upload pipeline	■ Fails on Windows	UnicodeEncodeError

Feature	Status	Evidence
Auth/login/DB persistence	■ Not runnable	DB absent/misconfigured
Heatmap	■ Broken	wrong maps key + unauth API
Chatbot	■■ Partial	key mismatch + XSS
Rewards/Meta-Tokens	■ Broken+exploitable	self-mint; schema missing
RAG	■ Not implemented	none exists
Flipkart	■ Fake	detect-only
Demo mode	■■ Implemented but hidden	flag ignored by UI
Security	■ Vulnerable	see Phase 12

Critical bugs

1. **Windows UnicodeEncodeError crash** in compliance pipelines (compliance_copy.py:56 etc.).
2. **Country-of-origin cross-validation false positive** ('us' in address matches "NutriPlus").
3. **Self-service token minting** (/api/gifts/add-tokens) — reward economy exploitable.
4. **Schema/code drift** — mt_tokens, gifts*, product_json_raw referenced but not in schema

(multiple endpoints break).

5. **Output-traceback leakage** to clients.
6. **Frontend ignores demo_mode** — placeholder shown as real; demo reports everything "ready for upload".
7. **No DB at all on target** + .env credentials stale (3307 vs 3306) + documented

database_schema.sql missing.

8. **Free-tier Gemini 429 crash** (no retry) after ~6 analyses.

Major bugs

- None-slice crash in analyze_product_data.
- Auth key mismatch login vs chatbot.
- Heatmap uses Gemini key as Maps key.
- /api/product/<id> IDOR (no ownership).
- Unauthenticated global-heatmap/image/gift-list (PIN leak).
- Unsalted SHA-256 passwords.

- No rate limiting / brute-force protection.
- Frontend auth via URL `?userId=&role=` (spoofable, logged).
- No XSS sanitization on chatbot render.

Minor bugs

- `popup.css` dead code; `database_schema.sql` startup doc link broken; landing video missing; CSS

`@import` ordering; duplicate `get_db_connection`; broken `__all__`; `__main__` typo; inconsistent grading scales across modules; `SCRAPER_*` envs undocumented impact.

Security vulnerabilities

See Phase 12 — highest: token self-mint, unauthenticated PIN/data exposure, open extension proxy, XSS, prompt injection, weak password hashing, plaintext DB creds + live API key on disk, IDOR, traceback leak.

AI/RAG reliability issues

1. **RAG doesn't exist** — nothing grounds decisions; rules are hardcoded dicts in prompts.
2. **LLM can invent rules/reasons** — no authority verification.
3. **Deterministic cross-validation produces false positives** (Bug #2) — blockchain-grade reproducibility absent.
4. Multiple Gemini calls per analysis → slow + quota-bound; no splitting/caching.

Features only simulated/demo

- Demo-mode compliance reports (fixed 70/B), when `GOOGLE_API_KEY` absent — **and these are presented as real**.
- "Rule retrieval" (there is no retrieval).
- Meta-Token economy (broken/exploitable).

Features genuinely production-ready

- **Amazon web scraper** (fetcher + extraction) — robust, real, works.
- **Gemini OCR + analysis wiring** — genuinely functional.
- **Scraper/extension Amazon product-page detection + overlay** — works.

Features in docs but NOT in implementation

- **RAG / semantic rule retrieval** — absent.
- **Flipkart support** — detection only.
- **Rewards/gift-table persistence** — tables don't exist.
- **Working heatmap** — broken.
- `database_schema.sql` — file doesn't exist.

Top 10 fixes before a hackathon/demo

1. Fix Windows Unicode crash (make `log()` ASCII-safe / set UTF-8).
2. Fix country-of-origin substring bug (`'us'` → word-boundary matching).
3. Load MySQL correctly + bring schema in sync (add `mt_tokens`, `gifts*`, `product_json_raw`), and create a real `database_schema.sql`.
4. Harden `/api/gifts/add-tokens` (server-side earning only) and require auth on `gift-list/global-heatmap/image`.
5. Stop leaking tracebacks: return sanitized errors; add structured logging.
6. Surface `demo_mode` in the frontend+extension; in demo mode never claim "ready for upload".
7. Add a Gemini retry/backoff + use the paid tier for demo (free tier dies after ~6 runs).
8. Implement real ownership checks (fix IDOR) and switch to `bcrypt/argon2`.
9. Fix chatbot `localStorage` key mismatch; sanitize chatbot HTML (use a markdown renderer, no `dangerouslySetInnerHTML`).
10. Provide a correct Google Maps key and hide the Gemini key from the client; delimit the extension proxy's allowed URLs.

Top 5 improvements to stand out

1. **Add a real, authoritative Legal Metrology rules database with citation-backed grounding** (replace hardcoded prompt dicts) — the single biggest credibility win.
2. **Implement prompt-injection/input sanitization + LLM-output validation** so adversarial product

data can't skew verdicts.

3. **Split analyses into a fast OCR pass + cached rule engine** to cut latency and quota use.
4. **Ship a one-command setup** (docker-compose or a setup script) so judges can actually run it — the #1 blocker today.
5. **Tighten the Chrome extension** (scoped permissions, no open proxy, true Flipkart support, demo/sim badges) for a polished, trustworthy demo.

Bottom line: Can you demonstrate MetaMark and claim it works? **Only partially.** The **scraper and Gemini engine are genuinely real** — but I would **not** demo the full compliance verdict, the seller upload, the heatmap, or the rewards as working, because I proved each of those is either broken, falsely positive, or an exploit. To be confident in front of judges, fix the Top 10 first — most importantly the Windows crash, the false-positive compliance bug, the missing database, and Demo Mode being shown as real.